Assignment 4

Relational Database vs. NoSQL Database

Comparative report with use cases and technology selection

Project: LibSphere - Library Management System **Prepared:** 23 October 2026

Platform assumption: MySQL 8.0.16 or later (InnoDB). Outputs in this report are expected results derived from the included sample dataset; run the SQL in MySQL Workbench to capture authentic execution screenshots if required by the instructor.

Relational and NoSQL databases solve overlapping but different problems. A relational database organizes data into related tables with a defined schema and SQL. NoSQL is a broad family that includes document, key-value, wide-column, and graph databases; behavior varies by product, so it is inaccurate to assume that every NoSQL database sacrifices consistency or that every relational system scales only vertically.

1. Comparison

Factor	Relational (SQL)	NoSQL
Data model	Tables, rows, typed columns, keys and relationships.	Documents, key-value pairs, wide columns, or graph structures; often tuned to access patterns.
Schema	Usually defined explicitly and changed through planned migrations.	Often flexible; validation can still be applied, but records may have different shapes.
Scalability	Vertical scaling is common; horizontal partitioning/replication is supported but can require careful design.	Many systems are designed for horizontal scale-out and partitioning across nodes.
Consistency	Strong transactional consistency and constraints are common in OLTP systems; actual behavior depends on configuration and topology.	Varies by product. Some support ACID transactions; others emphasize availability and tunable/eventual consistency.
Performance	Excellent for transactions, indexes, joins, and complex ad hoc reporting when modeled well.	Can be very fast for predictable key lookups, document reads, high-volume writes, or data-local access patterns.
Relationships	Foreign keys and joins are first-class capabilities.	Relationships are often embedded or referenced; joins may be limited, costly, or not the main modeling strategy.
Integrity	PK, FK, UNIQUE, CHECK and NOT NULL constraints can enforce many rules in the database.	Typically relies more on application logic or database-specific validation; capabilities vary.
Common fit	Banking, reservations, inventory, ERP, library loans, payroll, and reporting.	Shopping carts, session stores, flexible product catalogs, telemetry, activity events, and globally distributed workloads.

2. Real-world examples

Example application	Suitable technology pattern	Reason
Library circulation and borrowing	Relational: MySQL or PostgreSQL	A loan must reference an existing member and book. Unique ISBN/email values, checkout/return transactions, and reports spanning several tables are central requirements.
Online store product catalog with variable attributes	Document database such as MongoDB	A camera, book, and clothing item can have different descriptive fields. Documents can store attributes together when they are commonly read together.
Shopping carts, web sessions, live voting	Key-value/document service such as Amazon DynamoDB	The access pattern is often a fast read/write by a known key, and managed horizontal scaling can reduce infrastructure work.
Large distributed event/telemetry stream	Wide-column store such as Apache Cassandra	A distributed architecture can support large write volumes and scale throughput by adding nodes; the model should be designed around query patterns.

These examples are architectural patterns, not rules that force one product on every organization. MongoDB supports transactions, DynamoDB supports ACID transactions, and consistency options vary across products. A proper selection evaluates the exact workload rather than the “SQL vs. NoSQL” label alone.

3. How organizations choose

- **Data correctness and auditability:** choose a database that can enforce key invariants and recover complete transactions when several related records must change together.

- **Query shape:** frequent joins and ad hoc reporting tend to favor relational modeling; access patterns dominated by a key or self-contained document may favor NoSQL.
- **Growth and availability:** estimate throughput, latency, geography, acceptable downtime, replication, and recovery goals before choosing an architecture.
- **Operations and cost:** compare managed-service charges, database skills, backups, monitoring, licensing, migrations, and the cost of failures or inconsistent data.
- **Change rate:** evolving or heterogeneous attributes may favor a flexible document model, while highly governed and interconnected data often benefits from explicit relational structure.

4. Recommendation for LibSphere

Use a relational database as the system of record. Books, authors, categories, members, and loans are strongly related entities. Foreign keys protect circulation records from orphaned references; UNIQUE and CHECK constraints protect core rules; JOINs support overdue-loan lists, member borrowing histories, and inventory reports. A NoSQL store could be added later for search logs, clickstream analytics, or flexible recommendation metadata, but it should not replace the canonical loan ledger without a careful consistency design.

References

• MongoDB Manual, “What is MongoDB?” and “Data Modeling in MongoDB”: https://www.mongodb.com/docs/manual/ and https://www.mongodb.com/docs/manual/data-modeling/

• Amazon Web Services, “Amazon DynamoDB Documentation” and use cases: https://aws.amazon.com/documentation-overview/dynamodb/ and https://docs.aws.amazon.com/whitepapers/latest/big-data-analytics-options/amazon-dynamodb.html

• Apache Cassandra Documentation, “Cassandra Basics” and architecture overview: https://cassandra.apache.org/_/cassandra-basics.html and https://cassandra.apache.org/doc/latest/cassandra/architecture/overview.html

• MySQL Reference Manual, “FOREIGN KEY Constraints”: https://dev.mysql.com/doc/refman/8.0/en/constraint-foreign-key.html